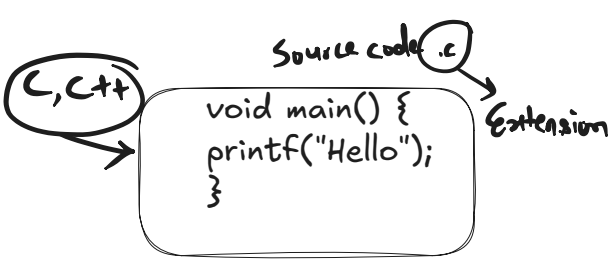
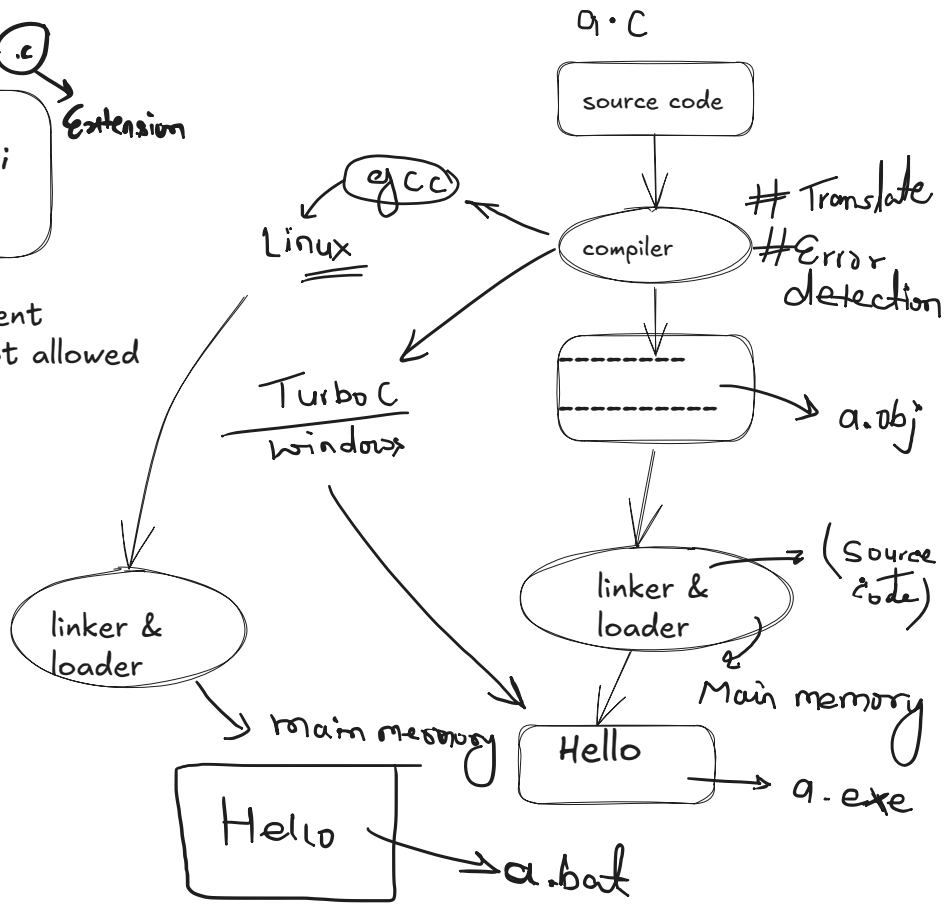
JAVA

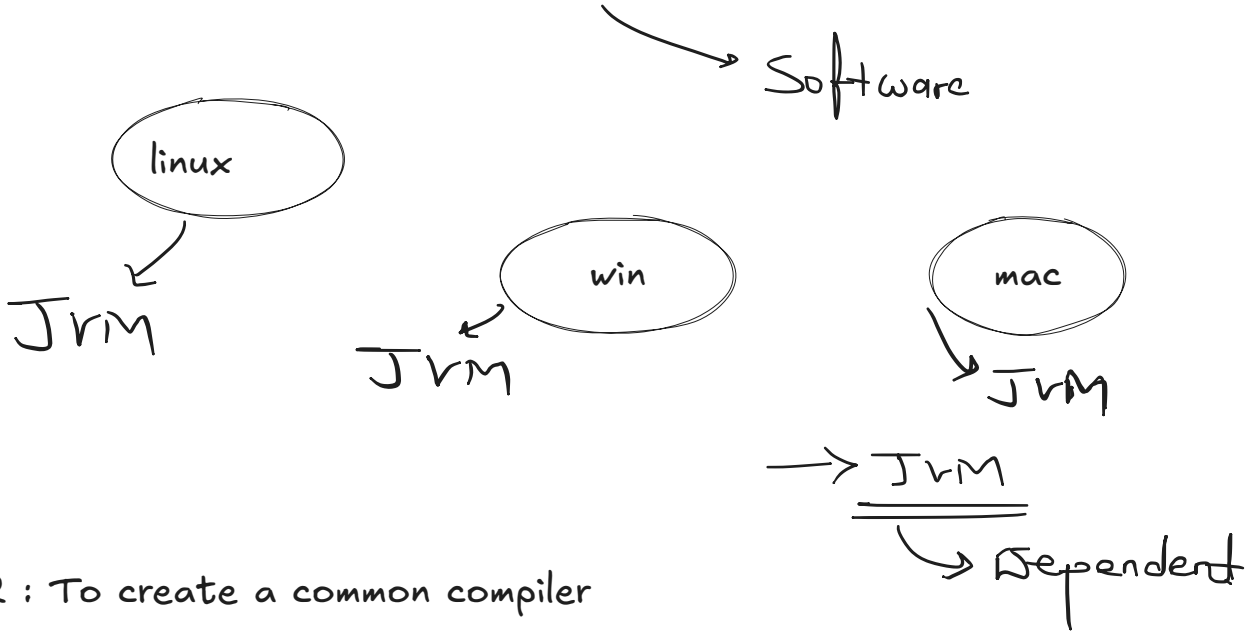
Need Of Java



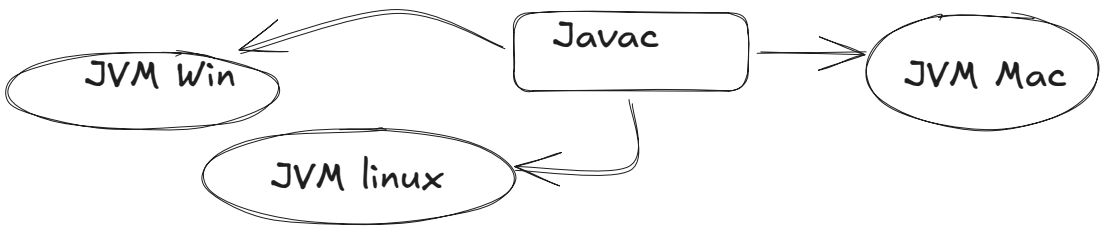
C, C++ --> Platform dependent
C, C++ --> Cross Cutting not allowed



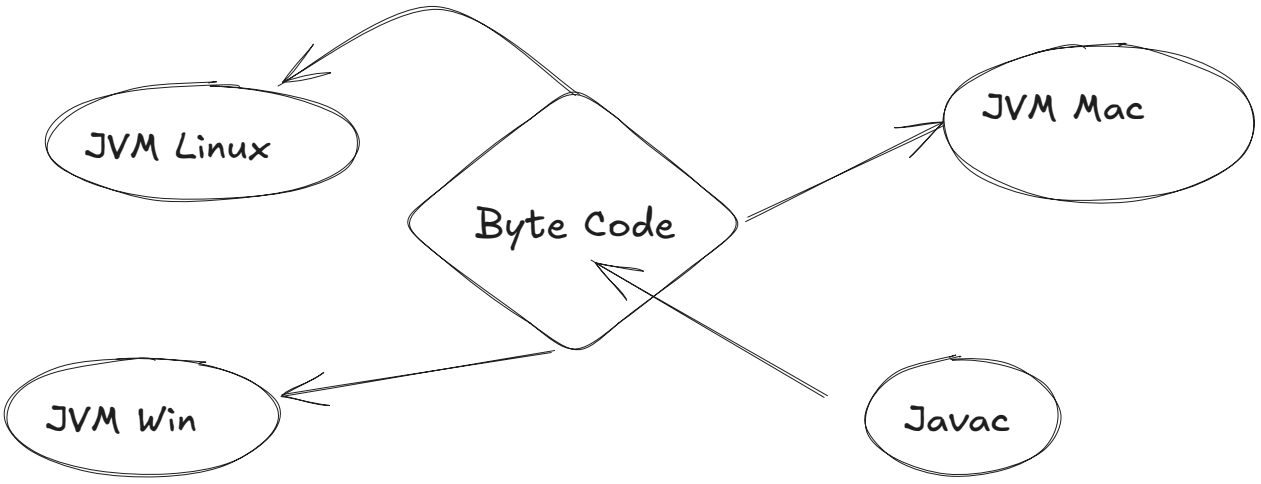
How to resolve this concept :
Step 1 : To create a JVM(Java Virtual Machine)



Step 2 : To create a common compiler

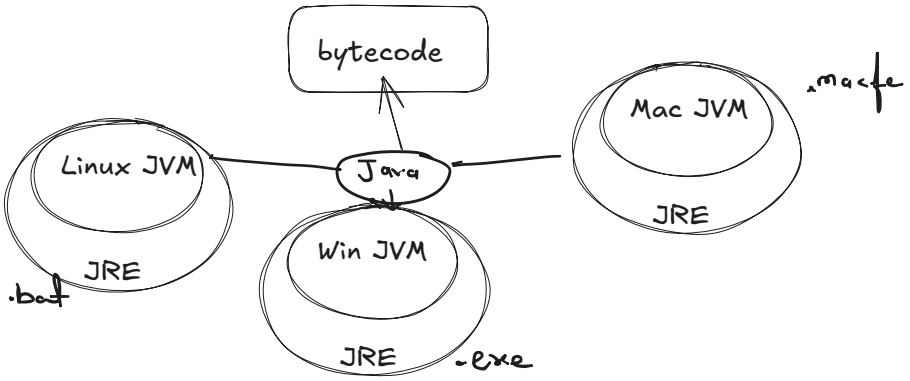


Step 3 : To create a common file (byte code)



#Java Compiler compile the Java Source file
and change in bytecode.

Step 4 : To create an Interpreter (Java) to bytecode verification.

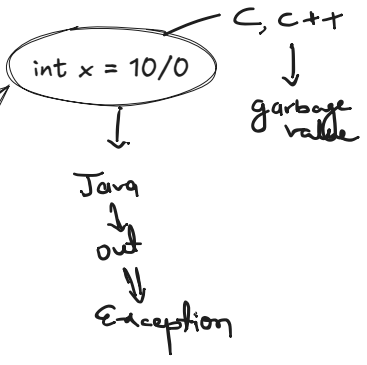


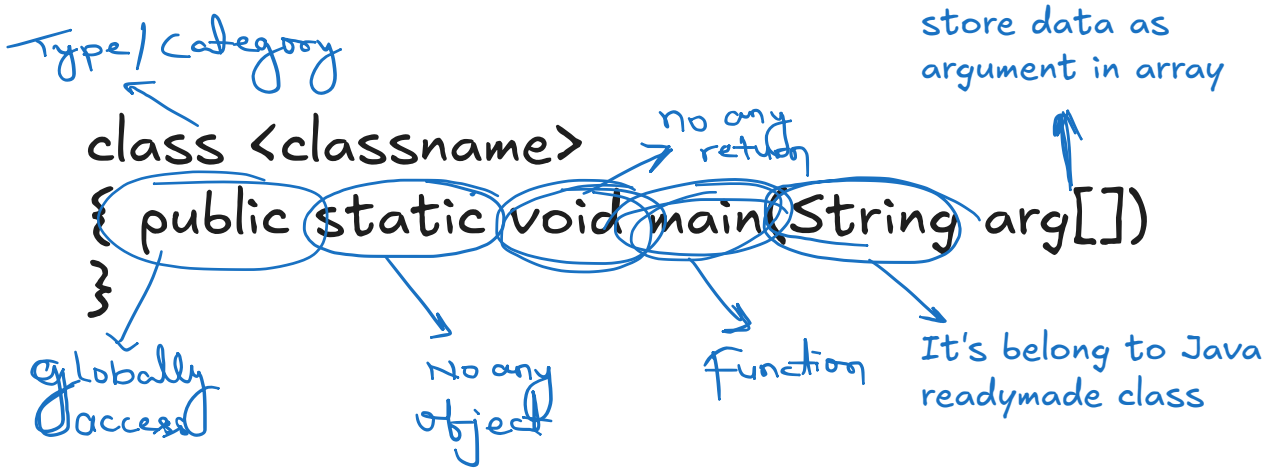
JVM, JRE --> Platform dependent

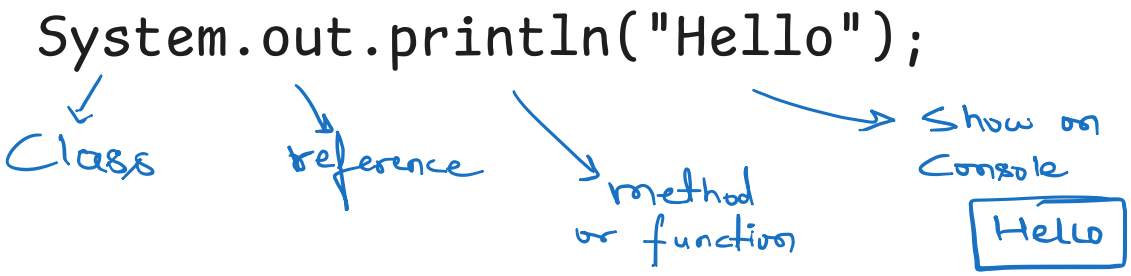
Software

Introduction to Java

--> 1995
--> James Gosling
--> Object-Oriented
--> Simple Programming Language
--> Platform Independent
--> Secured
--> Robust
--> Portable
--> Multi-thread







Step 1 : To Compile the Java Program
javac <filename>.java

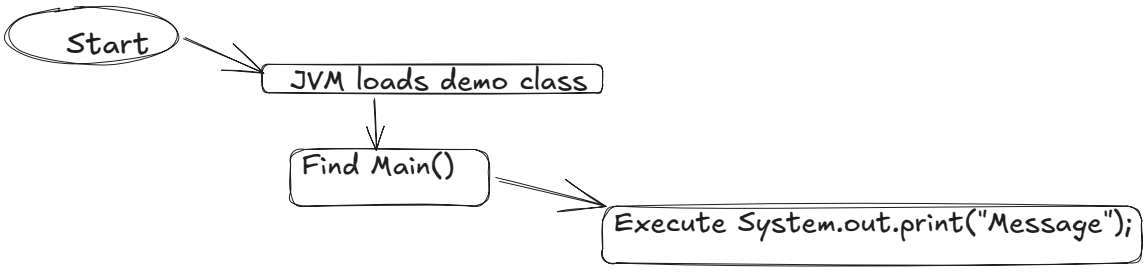
Step 2 : To execute the java Program
java <classname>

(Class is a type or categorisation)

public --> JVM can access the method

static --> JVM can call it without creating an object
void --> Method doesn't return any value
main --> Special method recognized by the JVM
String arg[] --> Store commandline argument

~ How the program execute

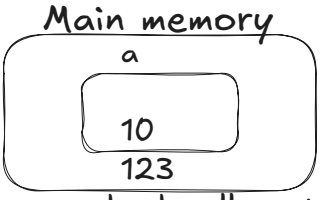


Variable and Data Type

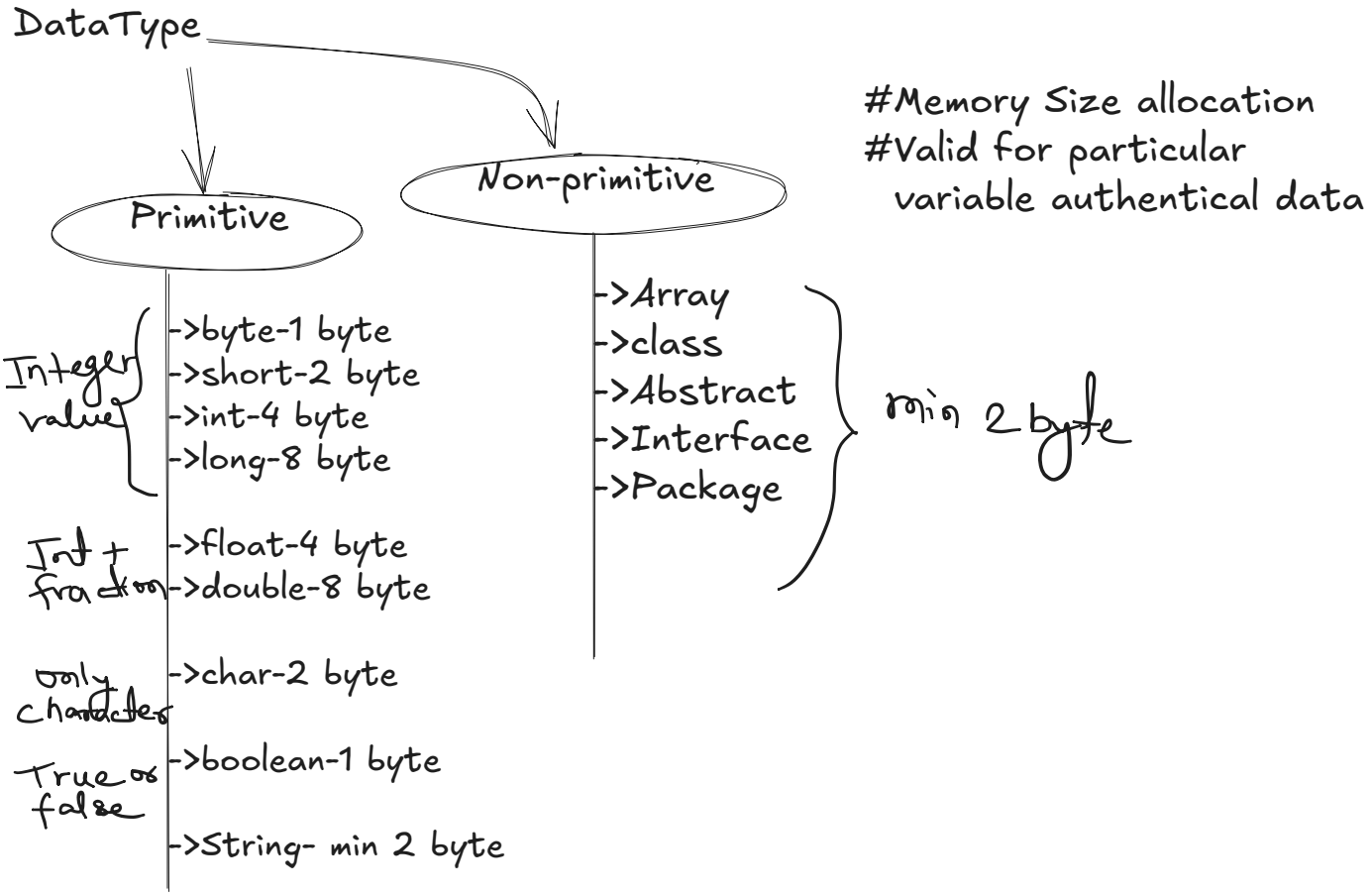
Variable :- datatype variablename = value/exp //Runtime

Optional

a = 10



--> Variable is a name location which allocates to a main memory and store data to it during runtime.



--> Type Casting : Type casting means converting a value from one datatype to another datatype.

Types of casting

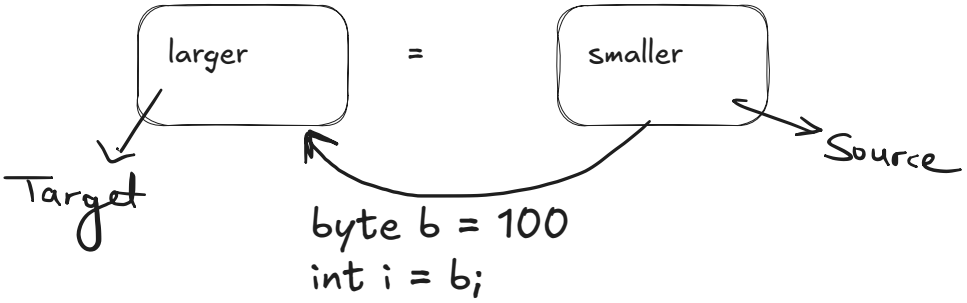
There are mainly two types :

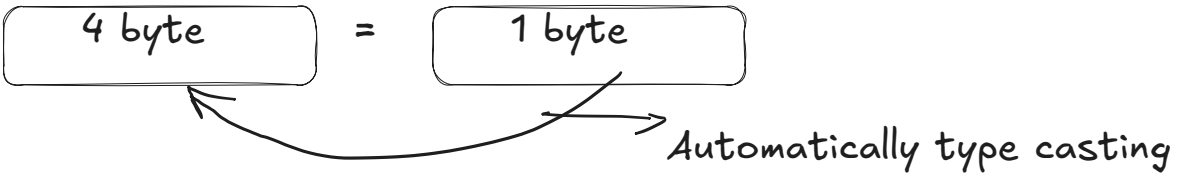
1. Widening Casting (Implicit casting)
2. Narrowing Casting (Explicit casting)

1. Widening Casting

Converting a smaller data type into a large data type automatically.

order : byte --> short --> int --> long --> float--> double



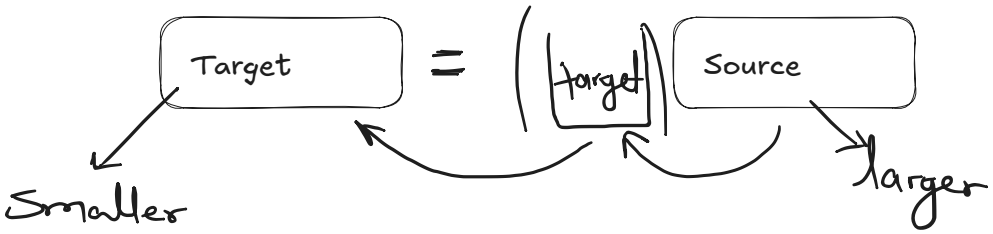


2. Narrowing casting (Explicit)

Converting a larger data type into a smaller data type.

double --> float --> long --> int --> short --> byte

Formulas : -

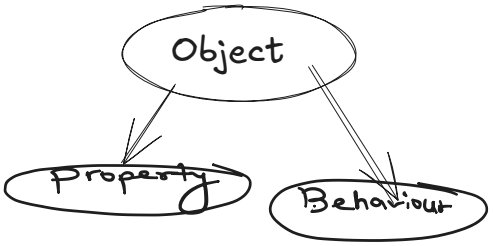


OOP's

--> OOPS in Java

Real World Entity

OOPS (Object Oriented Programming System) is a programming approach where we organize program using object and class.



--> It is a real world entity.

--> OOPS Concept

1. Class
2. Object
3. Main Piller of OOPS
    1. Encapsulation
    2. Polymorphism
    3. Inheritance
    4. Abstraction

1. Class

```
class car{
    //member data
    String color;
    String model;

    //member function
    void drive(){
          System.out.println("Car is running");
           }
```

A class is a blueprint or template for creating object.

2. Object

```
car c = new car();
c.color = "red";
c.model = "xb";
c.drive();
```

An object is a real instance of a class.

3. Main Piller

Encapsulation :

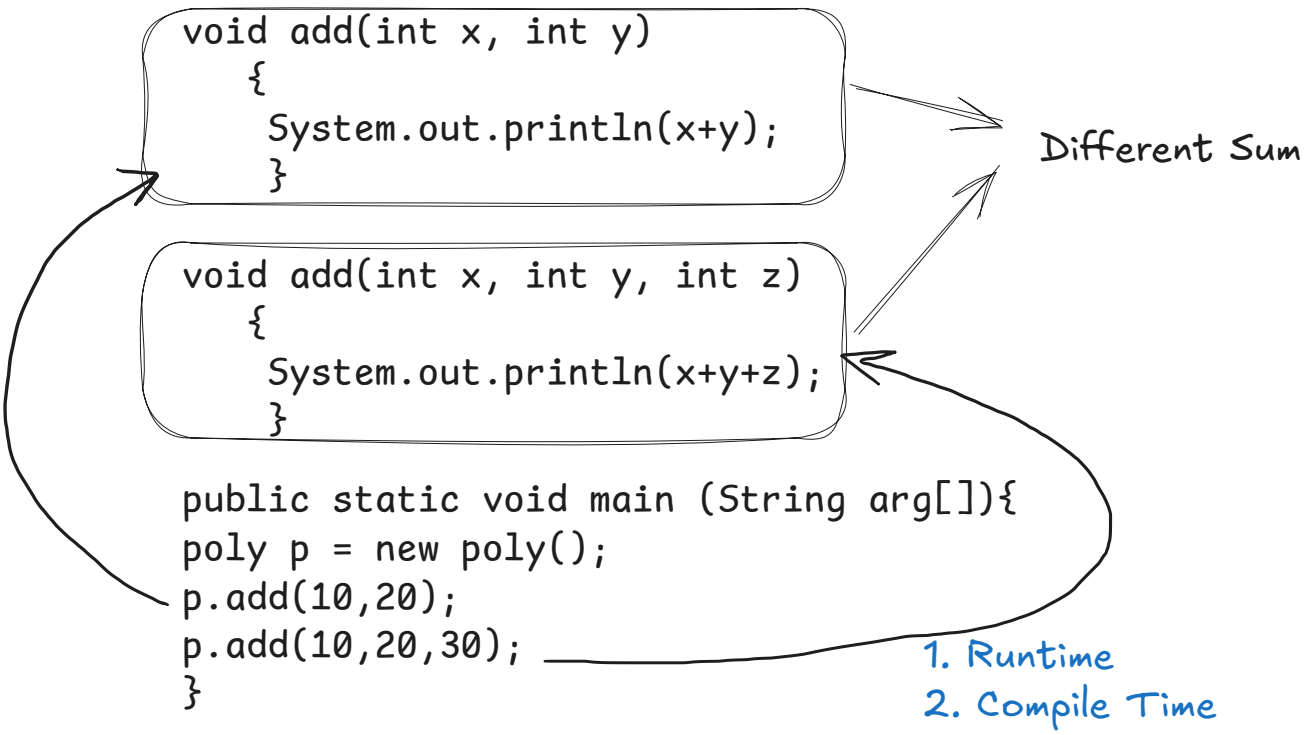
```
class BankAccount{
    private double balance;
    public void setBalance(double balance){
        balance = balance;
                }
           public double getBalance(){
                return balance;
                    }
                }
```

Encapsulation means wrapping data and methods into single unit (class) and protecting data using private.

--> Benefits
- Data Security
- Controller Access

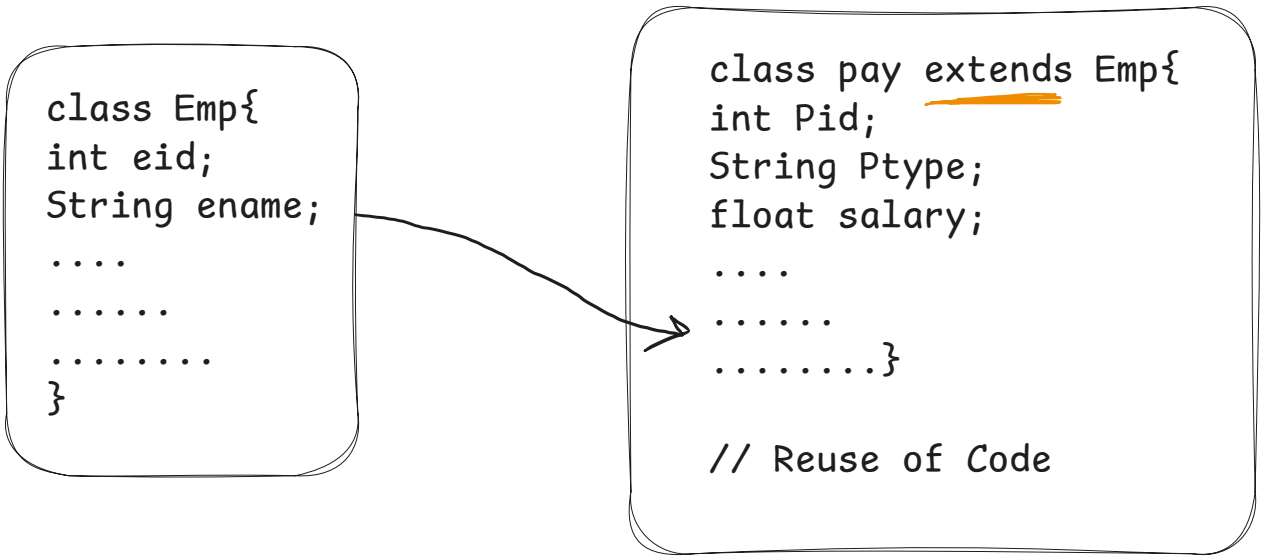
Polymorphism : It allows the same method name to behave differently.

Poly + mor + phism



Inheritance : One class can acquire the properties and methods of another class.

--> We use the extends keyword.



Abstraction: Hiding unnecessary implementation details and showing only the required functionality.

--> Java Provides abstraction mainly through :

- Abstract Class
- Interface

```
abstract class RBI(){
abstract void getInterestRate()
}
public class SBI extends RBI(){
void getInterestRate(){interestrate = 9%;}
}
public class CNRB extends RBI(){
void getInterestRate(){interestrate = 12%;}
}

//Abstract are achieve in structure
//Polymorphism
```

Access Modifiers in Java

Access modifiers is used in class, variable, method or constructor.

Types of Access Modifiers

1. Private
2. Default
3. Protected
4. Public

-> Private : A private member can be accussed only within the same class.

```
class student {
    private int marks = 90;
    void disp()
            {
    System.out.println(marks);
        }
//try to access it from another class
    class Main{
    public static void main(String arg[]){
            student s = new student();
            System.out.println(s.marks);
            }
        }
```

-> Default Access Modifier
When we do not specify any access modifiers, Java uses default access modifier.

```
class Student{
    int marks = 90; //default
    }
```

--> A default member can be accessed only within the same package.

```
class Test{
    public void static main(String arg[]){
            student s = new student();
            System.out.println(s.marks); //Allow
            } }
```

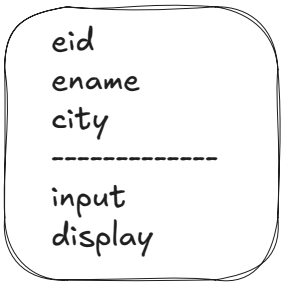
--> Protected Access Modifier :

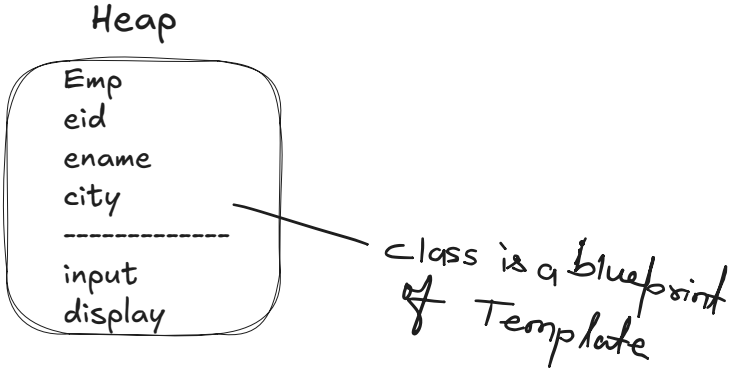
A protected member can be accessed

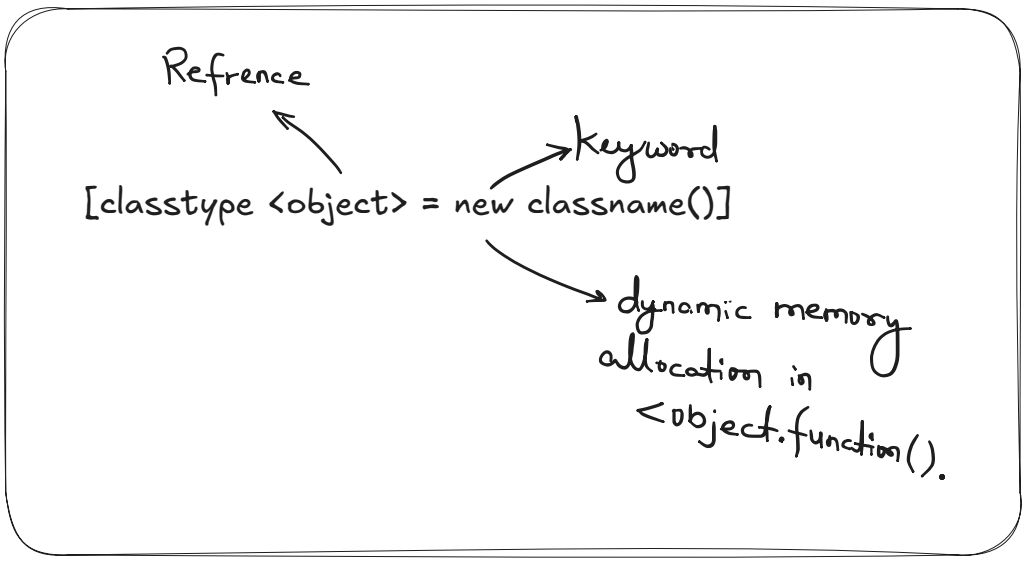
- Within same package
- In a subclass from another package

```
class Parent{
    protected int age = 90; //Protected
        }
class child extends Parent{
        public static void main(String arg[]){
            child c = new child();
            System.out.println(c.age); //Allowed
            }
```

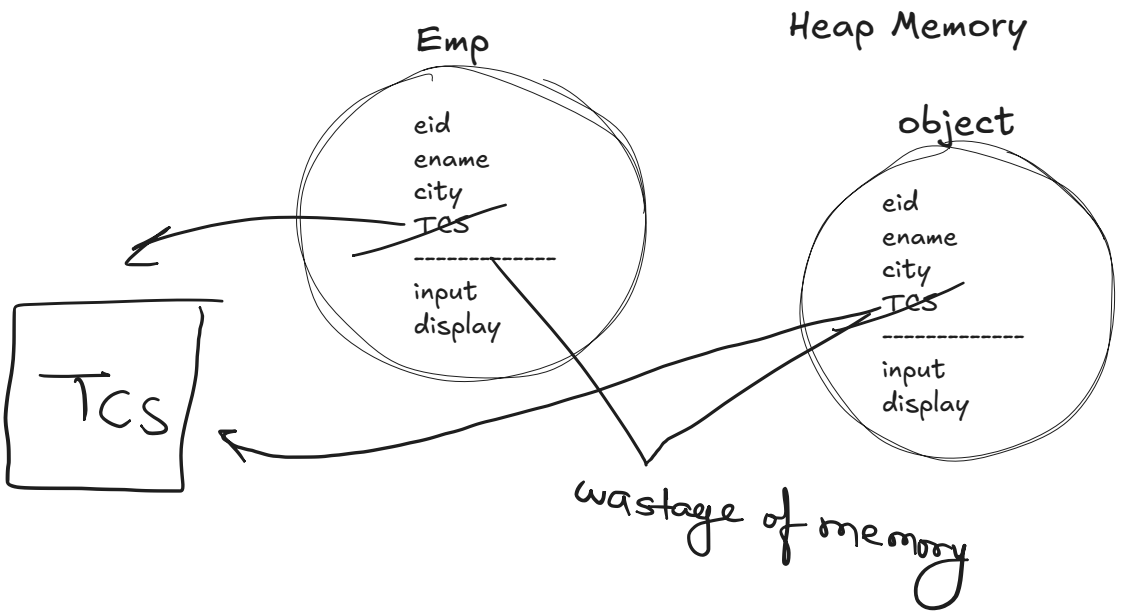
Q. What are the difference between class & object ?





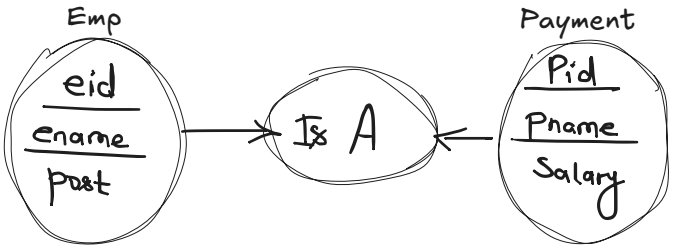


-> Need of Non-Static data member to change into static



#Static Compile time
#Static memory only
class identity

--> Inheritance in Java
Inheritance is an important concept of object oriented programming in Java.
--> It allow one class to acquire the properties and methods of another class.

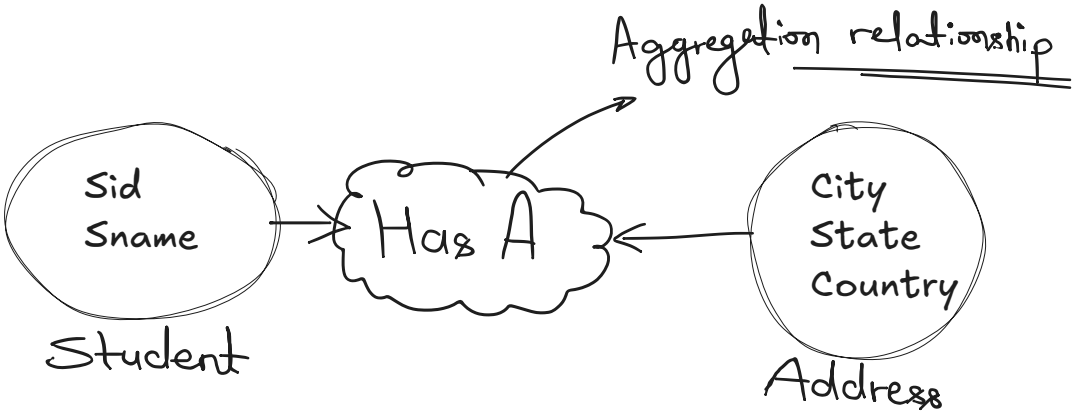



```
class Emp{
    int eid;
    String ename, Post;
    ---
    ---
    }
```

Indirect way

```
class Pay extends Emp{
    int Pid;
    String Payment;
    float Salary;
    ---
    ---
    ---
    }
```

reusability of code



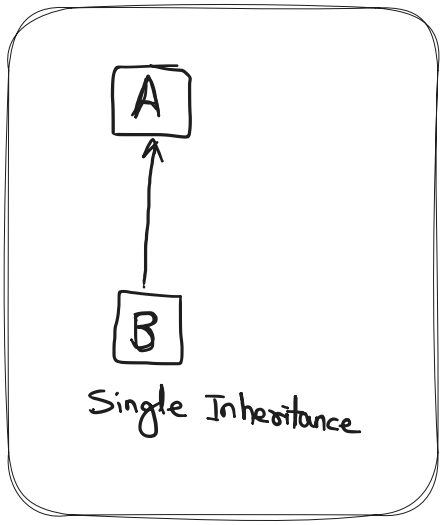

```
class Student{
    int Sid;
    String Sname;
    ---
    ---}
```

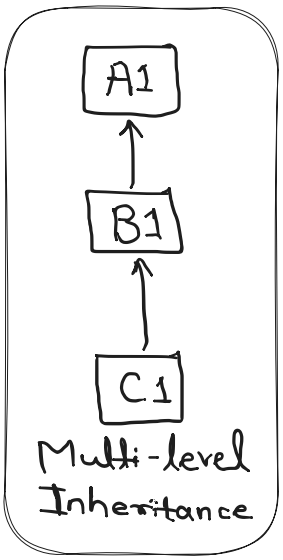
```
class Address{
   String city, state, country;
    int Sid;
    Student s; //Aggregation
}
```

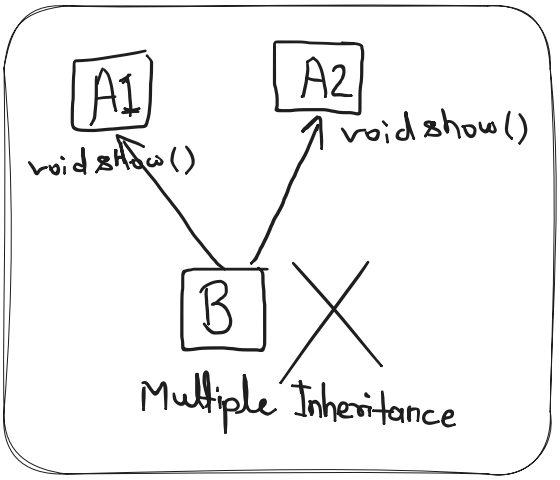
reference of Student class

Object of Student

--> Types of Inheritance in Java

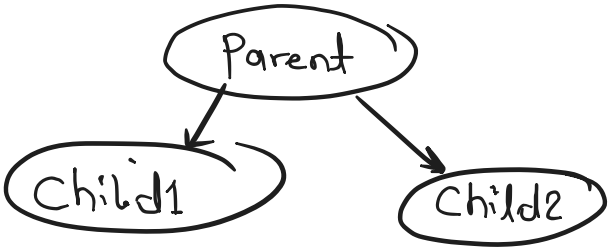




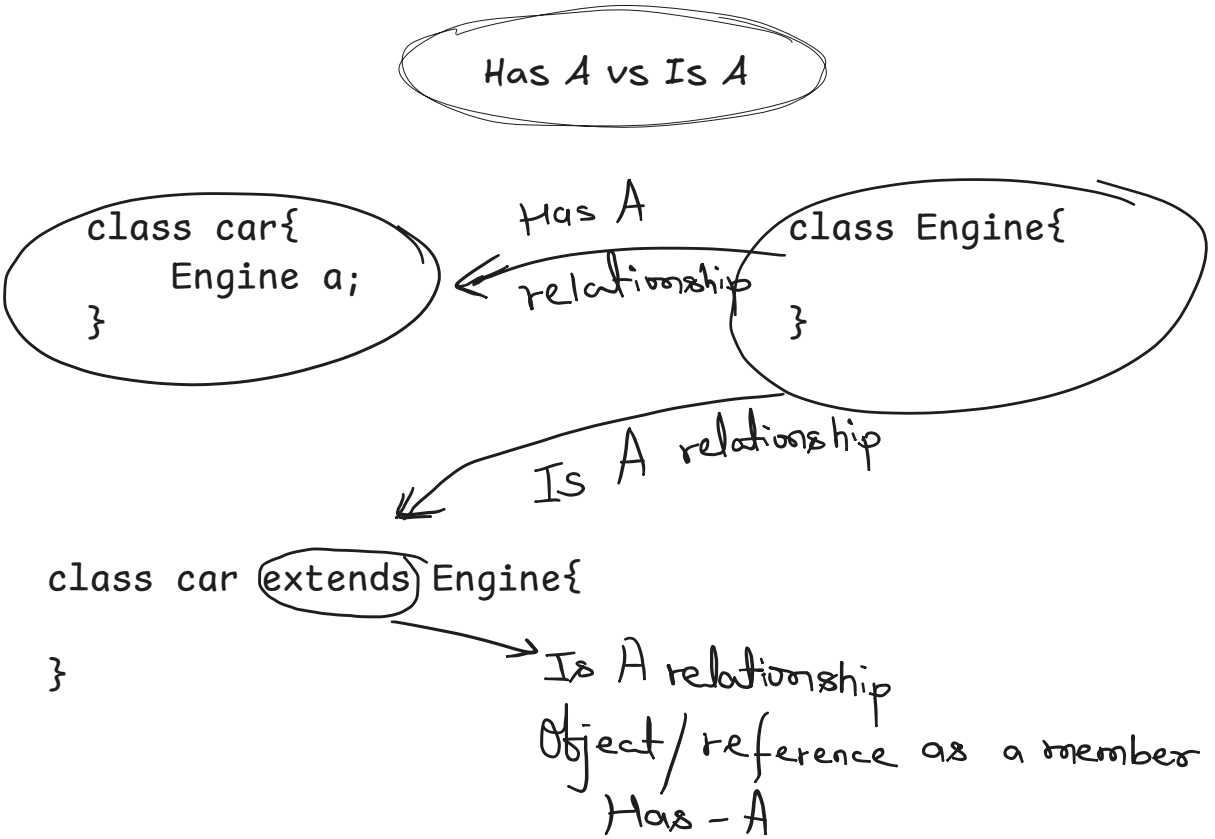


--> Private members can be inherit but not access in child class.

-> Hierarical Inheritance
It means one parent class is inherited by two or more child class.



--> Has A Relationship
It means one class contains a reference to another class as a member variable.
--> It represent composition and aggregation between the class.
--> Aggregation is type of has a relationship where one class contain a reference to another class but both object can exist independently.



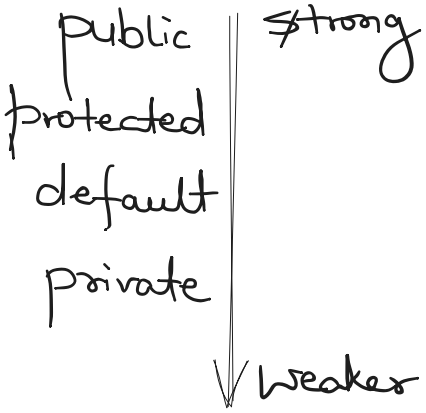
Composition --> In a composition, one class owns another class object, the contain object lifecycle is conceptually tight to the owner.

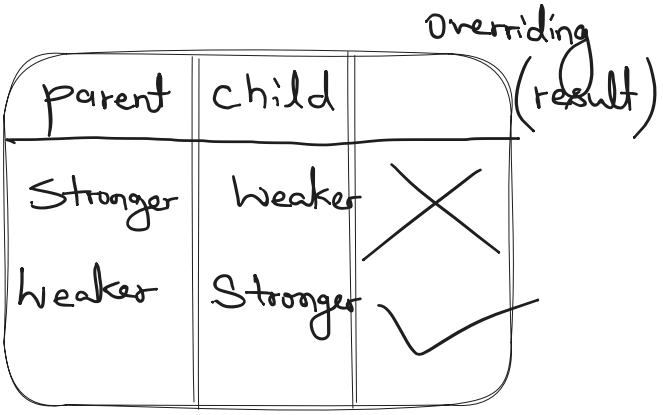
--> Method Overriding
Method overriding occurs when a child class provide it's own implementation of a method that is already defined in the parent class.

--> It is manually used to achieve Runtime Polymorphism.

-> Super Keyword in Java
The super keyword is used to refer to the immediate parent class object.

--> It is mainly used in three ways :
1. Call to parent class method
2. Call the parent class variable
3. Call the parent class constructor





--> Abstraction : Abstraction means hiding implementation details and showing only the essential features to the user.

Why do we use abstraction :

1. Hides implementation details
   user sees only what is necessary

2. Improve Security
   Internal logic is hidden from outside accessed.

3. Reduces complexity
   Makes Program easier to understand.

4. Provide Flexibility
   Implementation can be changed without effecting the user.

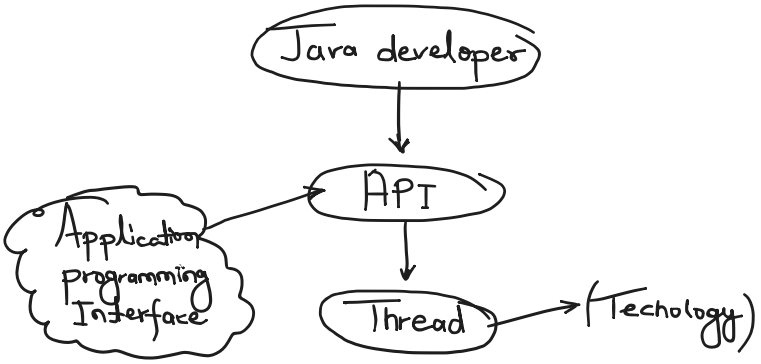
5. Supports loose coupling
   Code becomes easier to maintain and modify.

6. Help achieve standardization
   It defines what a class should do, not how it should do it.

--> How to achieve abstraction in Java ?

Java provides two main ways :-
- Abstraction class
- Interface

--> Interface in Java



--> Interface is nothing just like a contract, it contracts between java developers & technology, is known as Interface.

--> An Interface in Java is a blueprint, that defines a contract for classes.

Syntax :

```
interface <name_of_interface>
{
//body
}
class <class_name> implements <name_of_interface>{
//Implemented your interface
}
```



-->How Implements work

When a class uses an interface, we use the implements keyword.

--> Why do we use interface

1. Abstraction
   It hides implementation details and exposes only required behaviour.

2. Loose Coupling
   Interface reduce dependency between classes.

3. Multiple Inheritance
   Java doesn't allow a class to extend multiple classes but a class can implements multiple interface.

4. Achieve Polymorphism
   Same Interface reference, different implementation.

we can achieve 50% abst-
-raction through abstract
but 100% with interface

1. Default Methods

A default method is a method inside an interface that has a body and uses the default keyword.

Why do need it ?

Suppose an existing interface is implemented by 100 classes. If you add a new abstract method. Those classes may need to implements it. A default method provides an implementation so existing classes can continue working without adding their own implementation.

Static Method
A static method belongs to the interface itself.
you call it using the interface name.